

Лекция 6. Диффузионные модели и управляемая генерация изображений

Развёрнутый конспект на два академических часа. Дисциплина «Искусственный интеллект в креативных технологиях», магистратура 09.04.02, семестр 1

- Паспорт занятия
- Как читать конспект
- 1. Задача: научиться порождать то, чего не было
- 2. Прямой процесс и расписание шума
- 3. Обучение: сеть предсказывает шум
- 4. Обратный процесс: DDPM и DDIM
- 5. Цена шага и дистилляция
- 6. Архитектуры: от U-Net к трансформерам и потокам
- 7. Условие и безклассификаторное управление
- 8. Когда запроса недостаточно: пространственное условие и дообучение
- 9. Запрос и сравнение как эксперимент
- 10. Типичные заблуждения
- 11. Что делать в ЛР 9–10
- Итоговый квиз
- Выводы
- Источники и дата проверки

Паспорт занятия

Параметр	Содержание
Номер и тема	лекция 6 «Диффузионные модели и управляемая генерация изображений»
Место в курсе	семестр 1, шестая лекция из восьми; раскрывает механизм, который в лекции 4 использовался как «чёрный ящик» конвейера text-to-image; опирается на лекцию 5 (латентное пространство, автокодировщик) и на измерения ЛР 5–6; готовит ЛР 9–10
Длительность	2 академических часа (90 минут чистого времени)
Связанные ЛР	ЛР 9 «Промпт-инжиниринг»; ЛР 10 «Сравнительное исследование моделей text-to-image»
Результаты обучения	1) записывать прямой процесс и целевую функцию обучения и объяснять, что именно предсказывает сеть; 2) отличать вероятностный сэмплер от детерминированного и оценивать цену числа шагов; 3) объяснять безклассификаторное управление и находить разумное значение масштаба s по измерениям; 4) выбирать способ управления генерацией (запрос, пространственное условие, дообучение) по задаче; 5) планировать сравнение запросов и моделей как эксперимент с критерием, серией seed и допуском
Индикаторы компетенций	ПК-6.1 (протокол эксперимента); ПК-11.1 (безопасная и воспроизводимая работа с программными компонентами ИИ) — через наблюдаемые действия: студент записывает гипотезу и критерий до запуска, изменяет один фактор, сравнивает разность с разбросом между seed
Предварительные знания	лекции 1–5; ЛР 1–8

Как читать конспект

Конспект самодостаточен и читается без слайдов. Теорию сопровождают три группы чисел. Первая — арифметика по формулам расписания шума (значения пересчитываются за минуту). Вторая — демонстрационная диффузионная модель на плоскости (три перекрывающихся кластера, `lec06_demo.py`), обученная и выполненная в среде Engae 20.09.2026 на процессоре: на плоскости можно измерить всё, что на изображении можно только показать. Третья — замеры SD Turbo: в Google Colab на видеокарте Tesla T4 (ЛР 5–6, 19.09.2026) и в Engae на процессоре (20.09.2026). Значения помечены «демо», «Colab» или «Engae». Квиз из десяти вопросов помещён в конце; ключ с разбором есть только в версии для лектора.

Три сквозных вопроса: **что учит сеть** (шум, а не изображение), **чем платят за качество** (шагами, временем и отклонением от данных), **чем управлять** (запросом, пространственным условием или весами модели).

1. Задача: научиться порождать то, чего не было

Автокодировщик из лекции 5 порождает через одну точку латентного пространства и одно преобразование. У диффузионной модели иная схема: изображение строится длинной цепочкой мелких шагов, на каждом из которых сеть решает простую задачу — убрать немного шума. Простую задачу можно обучать устойчиво, а сложность возникает из числа повторений. Отсюда идут все свойства семейства: высокое качество и разнообразие (Но и др., arXiv:2006.11239), стоимость генерации, пропорциональная числу шагов, и удобство управления: условие можно подмешать на каждом шаге.

Для наглядности в лекции используются два масштаба. **Игрушечный**: точка на плоскости, три кластера с центрами на окружности радиуса 1,5 и разбросом 0,55; кластеры перекрываются, поэтому доля точек, которые ближе к чужому центру, ненулевая: у настоящих данных чистота класса равна 0,984. **Реальный**: SD Turbo, тот же механизм с латентным тензором $4 \times 64 \times 64$ (лекция 4).

2. Прямой процесс и расписание шума

Прямой процесс фиксирован и не содержит обучаемых параметров (запись формул — в лекции 4). Введём $\alpha_t = 1 - \beta_t$ и $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$; для сцены x_0 получаем $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$. Пара чисел $(\sqrt{\bar{\alpha}_t}, \sqrt{1 - \bar{\alpha}_t})$ показывает, из чего состоит x_t ; её удобно измерять **отношением сигнал/шум** по дисперсии

$$SNR(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad SNR_{\text{дБ}} = 10 \lg SNR.$$

Расписание из работы Но и др.: $T = 1000$, β_t растёт линейно от 10^{-4} до 0,02. Значения (арифметика по формулам, демо):

t	$\bar{\alpha}_t$	$\sqrt{\bar{\alpha}_t}$	SNR, дБ
1	0,9999	0,9999	40,0
50	0,971	0,985	15,3
100	0,897	0,947	9,4
250	0,524	0,724	0,4
500	0,0786	0,280	-10,7
750	0,00335	0,058	-24,7
1000	0,00004	0,006	-43,9

Доля сигнала по дисперсии падает ниже половины на шаге $t = 260$ и ниже одного процента на шаге $t = 674$; на последнем шаге остаётся 0,004 % — это уже практически

чистый гауссов шум, из которого стартует генерация. Замкнутая формула позволяет при обучении выбрать случайное t и сразу получить x_t , не проходя t промежуточных шагов.

Что видит сцена. Для сцены «диск» из ЛР 7 (32×32, три канала) шум с фиксированным seed 101 даёт PSNR между x_t и x_0 (арифметика, значения после обрезки в $[0, 1]$): 40,0 дБ при $t = 1$; 27,2 при 10; 16,2 при 50; 11,9 при 100; 8,0 при 250; 7,2 при 500 и дальше без изменений. Выход на 7,2 дБ означает, что от исходной сцены в x_t не осталось ничего измеримого; за первые 100 шагов PSNR падает с 40 до 12 дБ, то есть мелкие детали теряются в начале расписания, а вторая половина шагов работает уже с шумом.

Расписание — параметр модели. Слишком быстрое обнуление сигнала оставляет мало шагов для средних частот, слишком медленное расходует шаги впустую. Выбор расписания — часть проекта модели; для SD Turbo его определяет планировщик из конфигурации модели (лекция 4).

3. Обучение: сеть предсказывает шум

Сеть $\varepsilon_\theta(x_t, t, c)$ получает зашумлённый пример, номер шага и условие c и предсказывает добавленный шум ε . Цикл обучения короток: взять x_0 из набора, выбрать случайный t и шум ε , вычислить x_t , минимизировать

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, t, \varepsilon} \left[\left\| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, t, c) \right\|^2 \right].$$

По Но и др., эта простая цель связана со взвешенной вариационной границей и с сопоставлением оценок; для практики важно другое: цель — обычная среднеквадратичная регрессия, для которой не нужны состязательные сети и подбор баланса.

Демо. Сеть — полносвязная, 3 скрытых слоя по 256 нейронов, вход: точка (2 числа), признаки шага (17) и метка класса (3 значения плюс «пустое условие»); 8000 итераций по 512 примеров, метка заменяется пустой с вероятностью 0,1 (нужно для безклассификаторного управления, раздел 7). Обучение одной модели на процессоре Engage: 72,2; 68,2 и 73,5 с (три seed, 4 потока). Всего обучено три модели.

От шума к оценке чистого изображения. Из определения x_t следует оценка исходного x_0 по предсказанному шуму:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\hat{\varepsilon}}{\sqrt{\bar{\alpha}_t}}.$$

Демонстрация измерила ошибку такой оценки (RMSE по 4000 точкам, евклидова длина ошибки; демо, условие — известный класс). За эталоны взяты два тривиальных ответа: «не убирать шум» ($x_t/\sqrt{\bar{\alpha}_t}$) и «назвать центр класса» (0,779):

t	сеть	«не убирать шум»	центр класса
10	0,065	0,066	0,779
100	0,413	0,489	0,779
250	0,677	1,361	0,779
500	0,769	4,876	0,779
750	0,796	24,23	0,779
990	3,54	202,6	0,779

Вывод в три пункта. **Малый шум** ($t = 10$): ошибка сети совпадает с ошибкой «не убирать шум» (0,065 и 0,066), потому что убирать почти нечего. **Средний шум** ($t = 100\text{--}250$): сеть лучше обоих эталонов; при $t = 250$ ошибка 0,677 против 1,361 и 0,779. **Большой шум** ($t \geq 500$): ошибка сети (0,769 и 0,796) практически равна ошибке ответа «центр класса» (0,779) — сеть может сказать лишь, к какому кластеру принадлежит точка. Отсюда следует привычная закономерность генерации: на ранних шагах (большой шум) определяется грубая «композиция» (какой кластер), на поздних — детали внутри него; на изображениях её проверяют по промежуточным $\hat{x}_0$. На $t = 990$ ошибка оценки вырастает до 3,54: формула делит на $\sqrt{\bar{\alpha}_t} = 0,006$, и малая ошибка предсказания шума усиливается примерно в 150 раз; поэтому сэмплеры не доверяют $\hat{x}_0$ на самых больших t (причина проверялась только арифметикой, отдельного опыта нет).

Пример из жизни 1. Проявка фотографии. *Ситуация:* на листе в кювете изображение проступает постепенно: сначала крупные пятна, потом контуры, затем мелкие детали; фотограф знает, как долго держать лист, и «остановка проявки» задаёт результат. *Соответствие:* обратный процесс восстанавливает структуру от грубой к мелкой; номер шага сообщает сети, на какой стадии находится лист. *Граница аналогии:* химия проявки идёт сама и однозначно, а модель на каждом шаге сэмплирует из выученного распределения, и разные seed дают разные снимки.

4. Обратный процесс: DDPM и DDIM

Вероятностная схема (DDPM). На каждом шаге из x_t вычитается часть предсказанного шума, затем добавляется небольшая случайная добавка:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I), \quad \sigma_t^2 = \beta_t.$$

Нужны все $T = 1000$ вызовов сети. **Неявная схема (DDIM)** использует ту же обученную сеть, но строит детерминированную траекторию и позволяет пропускать шаги (Song и др., arXiv:2010.02502; по данным авторов, ускорение в 10–50 раз при небольшой потере качества). Один шаг DDIM с шага t на шаг $t' < t$:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}} \hat{\epsilon}, \quad \hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}.$$

Читается так: сначала оценить чистую сцену, затем «пересобрать» её с уровнем шума, соответствующим t' , взяв то же направление шума. Случайной добавки нет, поэтому одно и то же начальное x_T всегда даёт одно и то же изображение; на этом свойстве держится воспроизводимость по seed (лекция 4; у SD Turbo другой планировщик, но тоже без случайной добавки). Демо: два запуска DDIM с одним шумом на всех трёх моделях дали максимальное расхождение результатов 0,0.

Как измерить качество порождения. Нужна мера расстояния между распределением сгенерированных точек и настоящим. Используется энергетическое расстояние $E = 2 \mathbb{E} \|X - Y\| - \mathbb{E} \|X - X'\| - \mathbb{E} \|Y - Y'\|$ (чем меньше, тем ближе распределения; для одинаковых распределений в среднем ноль). Демо: сравнение 3000 сгенерированных точек (по 1000 на класс) с 3000 настоящими; для двух независимых выборок настоящих данных $E = -0,0006$ — это «пол шума» оценки. Эталонный вероятностный сэмплер DDPM, 1000 шагов: $E = -0,0006$, чистота класса 0,985 (настоящие данные — 0,984), 14,3 с на 3000 точек.

Число шагов DDIM. Демо (три модели, три seed сэмплирования, среднее по моделям и разброс s между ними; условие без управления):

Шагов	1	2	3	5	10	25	50	100
E	2,343	2,322	0,068	0,026	0,006	0,0006	-0,0002	-0,0005
s между моделями	0,492	0,487	0,003	0,0005	0,0004	0,0001	0,0001	0,0001
время, с на 3000 точек	0,016	0,030	0,040	0,065	0,125	0,316	0,716	1,374

Что видно. **Один–два шага** дают результат, далёкий от данных ($E \approx 2,3$, в 35 раз хуже, чем при трёх шагах). Причину мы не изолировали отдельным опытом; она согласуется с ошибкой $\hat{x}_0$ при большом t (раздел 3). **Три шага** дают $E = 0,068$, **10 шагов** — 0,006; при **25 шагах** $E = 0,0006$, при 50 и 100 шагах значения в пределах $\pm 0,0005$ от нуля, то есть в пределах порядка 0,001 от «пола шума» (-0,0006), который при 3000 точках не измеряется. Время растёт линейно: около 14 мс на шаг для 3000 точек. Итого DDIM с 50 шагами (0,72 с) даёт качество DDPM с 1000 шагами (14,3 с) при выигрыше по времени примерно в 20 раз; 25 шагов дают выигрыш в 45 раз.

Пример из жизни 2. Навигатор. *Ситуация:* маршрут можно записать тысячей поворотов по два метра, а можно двадцатью пятью крупными участками; человек приедет в то же место, если участки выбраны с учётом дороги. *Соответствие:* DDPM идёт по мелким шагам со случайными отклонениями, DDIM — по крупным детерминированным участкам, пользуясь тем же знанием дороги (сетью). *Граница аналогии:* слишком крупный участок (1–2 шага) уже не «маршрут», а прыжок; в демо он приводит к результату, далёкому от данных.

5. Цена шага и дистилляция

Время генерации складывается из числа шагов, стоимости одного вызова сети и постоянной части (кодировщик текста, декодер автокодировщика). Замер SD Turbo в Engage (процессор, 4 потока, float32, 512×512, генератор CPU, seed 101; повтор 1):

Шагов	1	2	4
время одного изображения, с	18,0	22,8	34,2

Три точки хорошо ложатся на прямую $\approx 12,3 + 5,4n$ секунд (отклонения не больше 0,4 с): **постоянная часть около 12 секунд, то есть две трети времени одного шага, а сам шаг сети — около 5 секунд.** Поэтому при одном шаге экономить на числе шагов уже нечего, а при 25–50 шагах цена шагов начинает определять всё: экстраполяция прямой (за пределами измеренного диапазона, оценка) даёт для 50 шагов около 4,7 минут (282 с) на изображение. Для сравнения в Colab на Tesla T4 один шаг при 512×512 занимал 0,34 с (JР 6); Engage на процессоре примерно в 50 раз медленнее. Размер кадра влияет почти пропорционально площади: 4,8 с при 256×256, 9,8 с при 384×384 и 18,0 с при 512×512 (73, 66 и 69 секунд на мегапиксель). Замер выполнен один раз, повторы внутри серии отличались не более чем на 0,7 с (кроме первого вызова с прогревом); для планирования лабораторной работы это ориентир, а не паспорт производительности. Пакеты `diffusers` и `transformers` в Engage пришлось ставить с ограничением версий (журнал замера): новые версии требуют более свежего PyTorch, чем установлен в среде.

Как сократить шаги без потери качества. Первый путь — детерминированные сэмплы (раздел 4): в демо 25 шагов вместо 1000 дали то же качество. Второй путь — **дистилляция**: «ученик» учится получать за 1–4 шага то, что «учитель» получает за десятки. SD Turbo обучен методом состязательной диффузионной дистилляции (Sauer и др., arXiv:2311.17042): по описанию авторов, сигнал от модели-учителя сочетается с состязательной потерей, которая удерживает качество при одном–двух шагах. Цена — у ученика нет обычного безклассификаторного режима (раздел 7), а лицензия и рекомендованный размер кадра определяются карточкой модели (лекция 4).

6. Архитектуры: от U-Net к трансформерам и потокам

Шумоподавляющая сеть — не фиксированный элемент схемы. Исторически это U-Net с механизмами внимания (лекция 3). Peebles и Xie (arXiv:2212.09748) заменили её трансформером над фрагментами латентного тензора (DiT) и показали, что качество растёт с вычислительной ёмкостью модели.

Второй сдвиг — в постановке задачи. **Сопоставление потоков** (Lipman и др., arXiv:2210.02747) обучает векторное поле, которое переносит шум в данные вдоль заданных путей; авторы подчёркивают обучение без симуляции процесса, а диффузионные пути — как частный случай. Esser и др. (arXiv:2403.03206) применили модель с выпрямленным потоком и трансформером с отдельными весами для текста и изображения (Stable Diffusion 3) и по своим измерениям получили лучшие результаты для генерации по

тексту, чем у диффузионных подходов. Для пользователя смена «двигателя» почти незаметна: остаются начальный шум, итерации, условие и декодирование; меняются расписание, число шагов, способ управления и требования к памяти. Поэтому сравнивать модели нужно по протоколу (раздел 9), а не по названию архитектуры.

7. Условие и безклассификаторное управление

Как условие попадает в сеть. В латентных моделях текст кодируется в последовательность векторов, а слои перекрёстного внимания дают каждому участку изображения свою смесь слов (Rombach и др., arXiv:2112.10752). Условие в демо — метка класса, поданная на вход сети.

Безклассификаторное управление (Ho, Salimans, arXiv:2207.12598). При обучении условие с вероятностью p заменяют «пустым» $\varnothing$ (в демо $p = 0, 1$); одна сеть учится и условному, и безусловному предсказанию. При генерации на каждом шаге вычисляют две оценки шума и усиливают их разность:

$$\tilde{\epsilon} = \epsilon_{\theta}(x_t, t, \varnothing) + s(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \varnothing)).$$

При $s = 1$ получаем обычную условную оценку; при $s > 1$ разность «условие минус пустое условие» усиливается. В обозначениях авторов веса $w = s - 1$. Смысл: разность оценок указывает, в какую сторону сдвигается шум при появлении условия, то есть куда нужно «подтолкнуть» образец, чтобы он лучше соответствовал условию; сдвиг можно истолковать как неявный классификатор, поэтому отдельная классифицирующая сеть не нужна. Цена — **два вызова сети на шаг** вместо одного (арифметика; у SD Turbo при $s = 3$ в ЛР 6 время выросло на 0,07 с, лекция 4).

Демо: что делает s . 50 шагов DDIM, три модели, три seed сэмплирования; условие — метка класса. **Чистота** — доля точек, ближайших к центру запрошенного класса (для настоящих данных 0,984 — кластеры перекрываются). **Разброс** — среднее расстояние до центра запрошенного класса (у настоящих данных 0,696). **E** — энергетическое расстояние до настоящих данных (раздел 4).

s	чистота	разброс	E
данные	0,984	0,696	—
1	0,986	0,649	−0,0002
2	1,000	0,638	0,063
3	1,000	0,856	0,165
5	1,000	1,391	0,483
8	1,000	2,343	1,162

Три наблюдения. **Первое.** При $s = 1$ сэмплы неотличимы от данных по E ; разброс на 7 % меньше настоящего (0,649 против 0,696; s по моделям 0,002), то есть модель чуть «узкая» уже без управления. **Второе.** Чистота достигает 1,000 уже при $s = 2$: для этой задачи управление полностью «исправляет» перекрытие кластеров. **Третье.** Дальнейшее

увеличение ничего не добавляет по чистоте, но точки уходят от данных: разброс растёт с 0,64 до 2,34 (при $s = 8$ — в 3,6 раза больше, чем при $s = 1$), E растёт с 0,063 до 1,162. На рисунке сэмплы при $s = 3$ сдвинуты от области перекрытия к внешним краям кластеров, а при $s = 8$ выходят за пределы области, где лежали данные. Так на плоскости проявляется то, что на изображениях называют пересыщением и артефактами: сильное управление уводит образец в область, где данных мало.

Осторожность. Числа относятся к игрушечной задаче с тремя перекрывающимися кластерами; порог «достаточного» s у реальной модели определяют по её метрикам (JР 10), а не переносят отсюда. У SD Turbo обычное управление выключено (лекция 4): значение из рекомендованного режима — 0,0.

Пример из жизни 3. Соль в супе. *Ситуация:* повар пробует блюдо без соли и с солью; разность вкусов — это вклад соли. Чтобы «подчеркнуть» вкус, он добавляет соли больше; сначала блюдо становится вкуснее, затем пересолено. *Соответствие:* разность оценок «с условием» и «без условия» усиливается коэффициентом s ; при умеренном s соответствие условию растёт, при большом — образец уходит в неестественную область. *Граница аналогии:* «вкус» здесь векторная величина в пространстве шума, и нельзя убрать лишнее, уменьшив s задним числом внутри одного прохода.

8. Когда запроса недостаточно: пространственное условие и дообучение

Пользователь различает три способа управлять результатом и выбирает наименьший достаточный.

Способ	Что задаёт	Когда применять	Цена
Запрос и параметры	смысл, стиль, композицию «в общих чертах»	быстрое исследование, серия вариантов	нельзя точно указать расположение и формы
Пространственное условие (ControlNet)	контуры, глубину, позу, границы, сегментацию	нужно сохранить композицию эскиза или позу	нужны карта-условие и модель управления
Дообучение малым числом параметров (LoRA)	стиль, персонаж, продукт	нужна устойчивая узнаваемость по многим изображениям	набор изображений с правами, время обучения, риск запоминания

ControlNet (Zhang и др., arXiv:2302.05543) закрепляет веса обученной модели и добавляет обучаемую копию блоков, соединённую «нулевыми свёртками»: в начале обучения их вклад равен нулю, поэтому возможности исходной модели не нарушаются, а пространственное условие вводится постепенно. **LoRA** (Hu и др., arXiv:2106.09685) замораживает веса и учит добавку к матрице $W \in \mathbb{R}^{d \times k}$ в виде произведения двух узких матриц: $W + BA$, где $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ и ранг $r \ll \min(d, k)$. Обучаемых параметров $r(d + k)$ вместо dk . Пример счёта для матрицы 1024×1024 и $r = 8$:

$8 \cdot 2048 = 16\,384$ вместо $1\,048\,576$, то есть 1,6 %. Авторы метода для GPT-3 175B приводят сокращение числа обучаемых параметров в 10 000 раз; для моделей изображений метод применяется по аналогии.

Пример из жизни 4. Трафарет и сменная оснастка. *Ситуация:* маляр либо кладёт трафарет (форма задана точно, цвет и фактура — его дело), либо ставит на инструмент сменную насадку, которая даёт узнаваемый рисунок валика, не меняя самого инструмента. *Соответствие:* пространственное условие задаёт форму, дообучение — устойчивый стиль; исходная модель остаётся прежней. *Граница аналогии:* насадка — это тоже обучение на изображениях, и права на них нужно подтвердить заранее (лекция 2); трафарет при слишком грубой карте-условии портит результат.

Вопросы прав на обучающие изображения и запоминания примеров обсуждаются в лекции 2; перед использованием дообученной модели результат проверяют на сходство с известными объектами. Названия авторов и защищённых персонажей как способ получить «их стиль» создают правовые и этические риски; решение о допустимости принимают до генерации.

9. Запрос и сравнение как эксперимент

Запрос — гипотеза. Её проверка требует критерия, заданного заранее, одного изменяемого элемента, серии seed и допуска (лекция 4, паспорт эксперимента). ЛР 9 закрепляет этот порядок в двух частях.

Часть 1: управляемая генерация изображения. Сравниваются формулировки запроса при неизменных модели, шагах, размере и seed; меняется один элемент (субъект, композиция, стиль, свет, отрицательный запрос). Разность считается существенной, только если превосходит разброс между seed. Оценка соответствия запросу измеряется метрикой; выбор метрики и её ограничения записываются в паспорт (лекция 4: CLIP измеряет соответствие текста и изображения, а не эстетику). **Что показал образец ЛР 9 (30 вариантов, Engae, SD Turbo, 512×512, 1 шаг, пять seed, CLIP ViT-L/14, 20.09.2026).** К базовому запросу из одного предмета добавлялась одна фраза. Значимый по критерию 2s рост сходства с добавленной фразой (clip_target) получен в 13 вариантах из 30: чаще всего для фраз о палитре и освещении (по 4 контекста из 5), реже для стиля и композиции (по 2 из 5); «заклинания качества» (masterpiece, best quality, highly detailed, 8k) не дали значимого роста ни в одном контексте из пяти, а энергия границ в среднем не выросла (−0,012); сходство с предметом (clip_subject) при любом добавлении в среднем снижалось (от −0,95 до −2,91): каждая добавленная фраза отодвигает изображение от предмета; отрицание внутри текста («no people, no text») в среднем не уменьшило, а слегка повысило сходство с текстом запрещённого объекта (+1,58; значимо в одном контексте). Демонстрационный вариант 1 (стилевая фраза «flat vector illustration») показал разность +4,61 при разбросе режима A 2,80 ($|Δ|/s = 1,6$: критерий не выполнен), хотя разность положительна на всех пяти seed и на картинке стиль сменился с фото на векторную графику. Вывод: один seed вводит в заблуждение, а критерий 2s по пяти seed консервативен; результат относится к этой модели, этим запросам и этой метрике.

Часть 2: запрос к языковой модели как звено конвейера. Схема: краткое описание заказчика → языковая модель по инструкции (роль, цель, ограничения, формат) → структурированное описание сцены (JSON с фиксированными полями) → шаблон запроса к модели изображений. Приёмы сравниваются на измеримом критерии: **zero-shot** — инструкция без примеров; **few-shot** — с несколькими примерами (Brown и др., arXiv:2005.14165: «Language Models are Few-Shot Learners»); **пошаговые инструкции** — просьба рассуждать по шагам (Wei и др., arXiv:2201.11903; Kojima и др., arXiv:2205.11916). Обзор Schulhoff и др. (arXiv:2406.06608) описывает 58 приёмов для текста и 40 для других модальностей и показывает, что промпт-инжиниринг — область с общей терминологией, а не набор «секретных фраз». Критерий для части 2 — проверяемый: например, доля ответов, которые проходят проверку схемы JSON, и доля обязательных полей, заполненных по правилам. Возможности структурированного вывода у поставщиков различаются; их проверяют по актуальной документации на дату работы (для GigaChat API структурированный вывод и вызов функций указаны в документации на 19.09.2026).

Что показал образец ЛР 9 (Perplexity, режим «Лучший», 20.09.2026). Для пяти описаний заказчика контекста 1 модели трижды (в новом чате каждый раз) был задан один запрос тремя приёмами; схема ответа содержала закрытые списки цветов и запретов, а в двух описаниях — ловушки («только не синий», «ровно два цвета»). Все девять ответов прошли проверку схемы ($K1 = 1,0$), передали смысловые элементы ($K2 = 1,0$) и точно перечислили запреты ($K3 = 1,0$): у сильной модели корректность упирается в потолок, и приёмы по ней не различаются. Различие нашлось в стабильности выбора полей *style* и *composition* между повторами ($K4$, доля совпадений по 30 сравнениям на приём): *few-shot* 0,93; *zero-shot* 0,77; *пошаговые инструкции* 0,50. Сравнения не независимы, выборка мала, режим модели — автовыбор: вывод предварительный и относится к этому сеансу; направление (примеры в запросе закрепляют выбор, пошаговый разбор его расшатывает) проверяется студентами в ЛР 9 на своих описаниях.

Сравнение моделей (ЛР 10). Чтобы сравнение имело смысл, при разных моделях фиксируется всё, что можно зафиксировать: набор запросов, число *seed*, размер кадра и устройство, а меняется одна вещь — модель или конфигурация. Что нужно записать: версию (ревизию) весов, планировщик, число шагов, масштаб управления, время и память на изображение (раздел 5), лицензию. Типичные ошибки сравнения: разное число шагов у разных моделей без указания; один *seed*; метрика, обученная на данных, близких к одной из моделей; сравнение времени на разном оборудовании. Вывод формулируется для этих моделей, этих запросов и этой метрики; универсального рейтинга не существует. **Что показал образец ЛР 10 (Engae, процессор, 20.09.2026).** Сравнивались четыре модели в заданных конфигурациях: SD Turbo (1 шаг), LCM Dreamshaper v7 (4 шага), bk-sdm-tiny (10 шагов DPM-Solver++; это сокращённый режим, в карточке — 50 шагов) и aMUSEd-256 (12 шагов, кадр 256×256, не диффузионная модель) — на пяти брифах и пяти *seed* (100 изображений; 30 вариантов = 5 брифов × 6 пар моделей). Сходство с брифом (*clip_brief*) почти не различает модели: средние по моделям 25,65–26,37, значимая по критерию 2s разность получена лишь в 9 вариантах из 30. Различают модели энергия границ (значимо в 21 варианте из 30; SD Turbo 0,051, LCM 0,011), контраст (в 24) и время на изображение: около 17 с у SD Turbo, 31–32 с у aMUSEd-256 и LCM, 65 с у bk-sdm-tiny.

В демонстрационном варианте (SD Turbo и LCM, бриф «афиша фестиваля робототехники») clip_brief модели не различил (разность +0,56, $|Δ|/s = 0,7$), а при слепом сравнении пяти пар автор пять раз выбрал SD Turbo; двусторонняя вероятность такого исхода при равенстве моделей равна 0,0625, то есть выше 0,05: метрика сходства с текстом не заменяет взгляд, а пять пар не доказывают предпочтение. Время одной и той же конфигурации в двух сеансах Engage различалось на 20 % и более, поэтому времена сравнивают только внутри одного запуска.

10. Типичные заблуждения

- **«Сеть предсказывает готовую картинку».** Сеть предсказывает добавленный шум; оценка чистого x_0 из неё при $t = 990$ в демо ошибается на 3,54, хуже эталона «назвать центр класса» (0,779), потому что ошибка усиливается делением на $\sqrt{\alpha_t}$.
- **«Больше шагов — всегда лучше».** В демо DDIM после 25 шагов не улучшается (E от 0,0006 до около нуля), а 1–2 шага дают $E \approx 2,3$: у числа шагов есть «колесо», и его находят измерением.
- **«DDPM качественнее DDIM».** В демо оба на уровне пола шума ($E = -0,0006$ у DDPM, $-0,0002$ у DDIM-50); различаются время (14,3 и 0,72 с) и воспроизводимость.
- **«Чем сильнее управление, тем точнее».** Чистота класса достигла 1,0 уже при $s = 2$; при $s = 8$ разброс вырос с 0,65 до 2,34, E — с 0,063 до 1,16.
- **«Управление можно включать всегда».** У дистиллированной SD Turbo обычное управление не предусмотрено (лекция 4).
- **«Длинный запрос заменяет пространственное условие».** Запрос задаёт смысл, но не точное расположение; для позы и композиции применяют ControlNet, для стабильного стиля — LoRA.
- **«Один удачный seed доказывает эффект формулировки».** Вывод делают по серии seed (лекция 1, раздел 6).

11. Что делать в ЛР 9–10

ЛР 9 «Промпт-инжиниринг». Часть 1: провести контролируемое сравнение формулировок запроса для SD Turbo (один изменяемый элемент, серия из пяти seed, критерий и допуск, записанные до запуска). Часть 2: сформировать структурированное описание сцены с помощью языковой модели и сравнить приёмы zero-shot, few-shot и пошаговых инструкций по проверяемому критерию. **ЛР 10 «Сравнительное исследование моделей text-to-image».** Сравнить модели или конфигурации по единому протоколу (раздел 9), записать паспорт запуска и вывод, ограниченный измеренным. Ориентир по времени из замера: одно изображение 512×512 при одном шаге в Engage занимает около 18 с, а в ЛР 10 время на изображение — от 17 с (SD Turbo) до 65 с (bk-sdm-tiny), один вариант выполняется около 7 минут; подробные условия и объём выборки указаны в методических указаниях. Каждая работа выполняется в отдельной папке среды.

Итоговый квиз

Десять вопросов идут по порядку разделов лекции, в каждом один верный ответ из четырёх. Подсказка направляет рассуждение, но ответа не называет; ключ с разбором есть в версии для лектора.

Вопрос 1. Нужно сохранить позу человека на эскизе и менять при этом стиль. Какой способ управления выбрать в первую очередь?

- A) пространственное условие (ControlNet)
- B) более длинный запрос
- C) увеличение числа шагов
- D) увеличение s до 8.

Подсказка: что из перечисленного задаёт расположение и форму?

Вопрос 2. Чему равно отношение сигнал/шум в дБ, если $\bar{\alpha}_t = 0,5$?

- A) 3 дБ
- B) -3 дБ
- C) 10 дБ
- D) 0 дБ.

Подсказка: чему равно $SNR = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$ и чему равен $10 \lg$ этого значения?

Вопрос 3. Что предсказывает сеть в упрощённой целевой функции?

- A) сразу чистое изображение x_0
- B) добавленный шум ε
- C) номер шага
- D) вес условия.

Подсказка: что стоит внутри нормы в $\mathcal{L}_{\text{simple}}$ рядом с ε_θ ?

Вопрос 4. Почему оценка $\hat{x}_0$ в демо при $t = 990$ (ошибка 3,54) хуже, чем при $t = 750$ (0,80)?

- A) сеть не обучалась при малых t
- B) шум на этом шаге не гауссов
- C) в формуле деление на $\sqrt{\bar{\alpha}_t} = 0,006$ усиливает малую ошибку предсказания шума примерно в 150 раз
- D) метка класса на этом шаге пропадает.

Подсказка: чему равно $1/0,006$?

Вопрос 5. Что верно для DDIM при одном и том же начальном шуме?

- A) результат детерминирован: два запуска совпадают (в демо расхождение 0,0)
- B) результат случаен на каждом шаге
- C) нужно всегда ровно 1000 шагов
- D) обученная сеть не нужна.

Подсказка: есть ли в шаге DDIM случайная добавка?

Вопрос 6. По таблице демо, каково наименьшее число шагов DDIM, при котором $E \leq 0,01$?

- A) 3
- B) 10
- C) 5
- D) 25.

Подсказка: E при 3, 5 и 10 шагах: 0,068; 0,026; 0,006.

Вопрос 7. DDIM-50 занял 0,72 с, DDPM-1000 — 14,3 с при неразличимом качестве. Во сколько раз DDIM быстрее?

- A) в 5 раз
- B) в 10 раз
- C) примерно в 20 раз
- D) в 50 раз.

Подсказка: 14,3/0,72.

Вопрос 8. Как формируется управляемая оценка шума при масштабе s ?

- A) $s \varepsilon_c$
- B) $\varepsilon_c + \varepsilon_\emptyset$
- C) $\varepsilon_\emptyset + s(\varepsilon_c - \varepsilon_\emptyset)$
- D) ε_c/s .

Подсказка: при $s = 1$ должна получиться обычная условная оценка.

Вопрос 9. Что показало демо при $s = 8$ по сравнению с $s = 2$?

- A) чистота выросла с 0,9 до 1,0
- B) время уменьшилось вдвое
- C) разброс уменьшился
- D) чистота осталась 1,0, а E выросло с 0,063 до 1,162: образцы ушли от данных.

Подсказка: сравните значения чистоты и E в таблице.

Вопрос 10. Сколько обучаемых параметров у LoRA для матрицы 1024×1024 при ранге $r = 8$?

- A) 16 384
- B) 8192
- C) 1 048 576
- D) 8.

Подсказка: $r(d + k)$.

Шкала самопроверки: 9–10 верных — тема освоена; 7–8 — повторить разделы 4 и 7; менее 7 — перечитать конспект и вернуться к ЛР 9–10.

Выводы

1. Диффузионная модель учит убирать шум: обучение — обычная регрессия по шуму, добавленному по известному расписанию; качество оценки чистого изображения падает при большом шуме.
2. Обратный процесс можно вести вероятностным (DDPM) или детерминированным (DDIM) способом; в демо 25–50 шагов DDIM дают качество 1000 шагов DDPM в 20–

- 45 раз быстрее, а 1–2 шага недостаточны.
3. Стоимость генерации складывается из шагов и постоянной части; в Engae для SD Turbo 512×512 около 12 с постоянной части и около 5 с на шаг; дистилляция переносит выигрыш во времени в саму модель.
 4. Безклассификаторное управление усиливает разность условной и безусловной оценок; в демо чистота насыщается при $s = 2$, а дальше растёт отклонение от данных.
 5. Запрос, пространственное условие и дообучение решают разные задачи; выбор делают по задаче, а не по популярности метода.
 6. Сравнение запросов и моделей — эксперимент: критерий и допуск до запуска, серия seed, один изменяемый фактор, паспорт запуска и вывод, ограниченный измеренным.

Источники и дата проверки

1. Denoising Diffusion Probabilistic Models — Ho J., Jain A., Abbeel P.; препринт arXiv:2006.11239 от 19.06.2020.
2. Denoising Diffusion Implicit Models — Song J., Meng C., Ermon S.; arXiv:2010.02502 от 06.10.2020.
3. High-Resolution Image Synthesis with Latent Diffusion Models — Rombach R. и др.; arXiv:2112.10752 от 20.12.2021.
4. Classifier-Free Diffusion Guidance — Ho J., Salimans T.; arXiv:2207.12598 от 26.07.2022.
5. Scalable Diffusion Models with Transformers — Peebles W., Xie S.; arXiv:2212.09748 от 19.12.2022.
6. Flow Matching for Generative Modeling — Lipman Y. и др.; arXiv:2210.02747 от 06.10.2022.
7. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis — Esser P. и др.; arXiv:2403.03206 от 05.03.2024.
8. Zhang L., Rao A., Agrawala M. Adding Conditional Control to Text-to-Image Diffusion Models (arXiv:2302.05543, 10.02.2023).
9. Hu E. и др. LoRA: Low-Rank Adaptation of Large Language Models (arXiv:2106.09685, 17.06.2021).
10. Adversarial Diffusion Distillation — Sauer A. и др.; arXiv:2311.17042 от 28.11.2023.
11. Brown T. и др. Language Models are Few-Shot Learners (arXiv:2005.14165, 28.05.2020); Wei J. и др. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models (arXiv:2201.11903, 28.01.2022); Kojima T. и др. Large Language Models are Zero-Shot Reasoners (arXiv:2205.11916, 24.05.2022).
12. Schulhoff S. и др. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques (arXiv:2406.06608, 06.06.2024).
13. Журналы выполнения: демонстрация `lec06_demo.py` и замер SD Turbo — Engae 26.8.2-НЗ, процессор, Python 3.11, torch 2.4.1+cu121, diffusers 0.32.2, transformers 4.57.6, 20.09.2026; JIP 5–6 — Google Colab, Tesla T4, 19.09.2026.

Публикации проверены по страницам препринтов 19.09.2026 (название, первый автор, дата и содержание аннотации); формулы — стандартные определения из указанных работ;

измерения — из журналов выполнения.